



YOU HAVE BEEN HANDED AN AGENT BEHAVIOUR POLICY

A draft behaviour policy for Claude Code on the web, with one repository attached.

It tells an agent, in files it can read, what it may do, what it may not, and what actually stands in the way of each thing. It was measured from the deployment rather than copied from a template, and it carries no score: it describes, it does not judge.

TEMPLATE

15 IT CAN DO · 6 WANTED · 9 NOT

7 UNBOUNDED

13 OF 15 MEASURED

[new to this? what an ABP is →](#)

01

Read the mandate, and correct it

It is a starting point written to be argued with. Most people authorised less than it assumes — and never mentioned the rest.

02

Give it to your agent

Copy the files into the agent's own context, download them as a zip, or hand over the PDF. Three ways, same files.

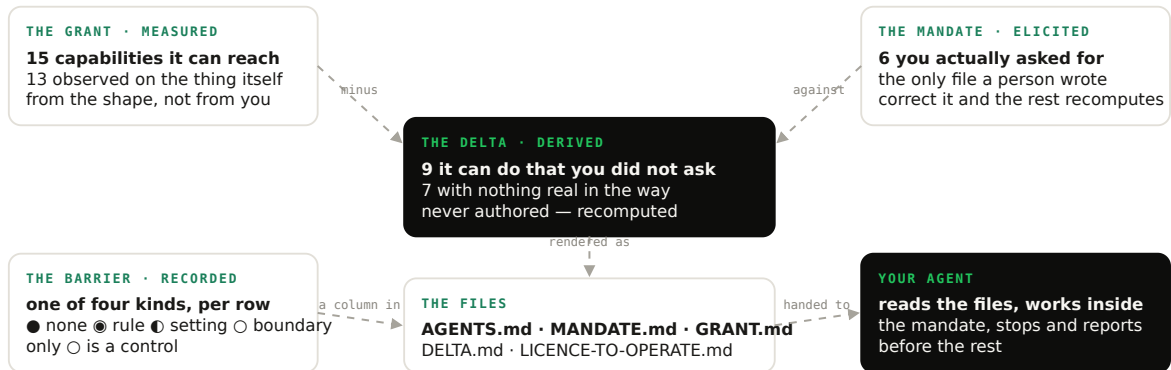
03

Sign the licence to operate

A named person, an interval, nine conditions each beside what enforces it. Until then the agent runs on nobody's say-so.

WHERE THE PIECES GO

Four objects, one route to the agent



Grant minus mandate is the delta. The barrier says what stands in the way of each row. The files carry all four to the agent – and the licence says who authorised it, until when.

THE WHOLE POLICY ON ONE SCREEN

What we want you to do. What we do not.

6 Here is what we want you to do

The mandate. Use these; this is why you were deployed.

<code>send.endpoint.allowed</code> Reach a permitted list of hosts	irreversible
<code>execute.process.host</code> Run programs as the account	undo with effort
<code>write.file.project</code> Change the project it is working on	undo with effort
<code>write.repository.project</code> Commit to the repository it was pointed at	undo with effort
<code>write.repository.tenant</code> Push to a code host (any branch it can reach)	undo with effort
<code>read.file.project</code> Read the project it is working on	undoable

6 of 6 wanted capabilities are in the grant. Nothing wanted is missing.

9 Here is what we do not want you to do

The excess: reachable, and not asked for. Each line says what actually enforces it.

`authenticate-as.credential.signing`

Sign commits with the key it holds

● nothing in the way

`delete.file.host`

Delete files anywhere the account can reach

● nothing in the way

`read.credential.host`

Read credentials stored where it runs

● nothing in the way

`read.file.host`

Read any file the account can reach

● nothing in the way

`read.record.history`

Read a retained record: shell history, past sessions

● nothing in the way

`authenticate-as.credential.tenant`

Act in accounts with the credentials it holds

○ a boundary – enforced

`write.file.host`

Change any file the account can reach

● nothing in the way

`create.schedule.tenant`

Create something that outlives the session, on the platform (a routine, a scheduled trigger, a new session)

● a setting – flippable

`create.schedule.host`

Create something that outlives the turn where it runs (a cron, a service)

○ a boundary – enforced

2 of 9 are enforced by a boundary. 7 are only asked — a rule in prose, a setting, or nothing. Those are what the licence accepts with its eyes open, and what a real control would close.

Not the person who made this? Start with the mandate anyway: it is the one file a person wrote, and it is short. Everything else in the vault is derived from it

and from the measured grant, and is rebuilt whenever either changes. If you are the agent: read AGENTS.md first. It is written for you.

What an Agent Behaviour Policy is, in five minutes

An Agent Behaviour Policy — ABP for short — is a written description, for one AI agent in one deployment, of everything it can do, what you authorised it to do, the gap between the two, and what actually stands in the way. It is written in files the agent itself can read, so it is both a record for people and an instruction set for the agent.

The problem it answers

When you connect an assistant to a repository, a mailbox, a drive or a cloud account, you know what you asked it to do. Almost nobody knows what it can do: the credential it runs on usually permits far more than the task needed, and nobody wrote that down. If the agent then does something you did not want, the first question you will be asked is what you did authorise. This document is the answer to that question, written before it is asked.

Four words, and the verb attached to each is the whole model

MEASURED

The grant

Everything the agent can actually reach — files, processes, credentials, network, repositories, schedules — measured from the deployment shape, not from your account and not by asking the agent.

ELICITED

The mandate

What you wanted it to do. You already know this; it is the one part a person writes. In a template it is a starting point, deliberately conservative, written to be corrected upward.

DERIVED

The delta

The grant minus the mandate: what it can do that you did not ask for. Never written by hand — recomputed whenever the grant or the mandate changes.

RECORDED

The barrier

For each capability, what stands in the way: nothing, a rule in prose, a setting the agent could flip, or a boundary enforced above it. Only the last one is a control.

Why there is no score

Every reader asks for one. The same policy is harmless where nothing valuable is reachable and dangerous with a production database attached tomorrow — and nothing about the document changed. A score needs the assets and the consequences, and this document has neither. It describes; a risk assessment, built on top of it, judges.

What it is not

- Not a guardrail: the files ask, the barrier column says what enforces.
- Not a compliance assessment, a certification or an audit of any product.
- Not a template: it is derived from one deployment shape, and a copy is corrected for yours.

What to do with this vault

- **If you run this agent:** read the mandate, correct it, give the files to the agent, and sign the licence. The three buttons on Start here are those steps.
- **If you were sent this by someone who runs it:** the Start here view is the whole policy on one screen; the Delta view is the part worth a conversation.
- **If you are the agent:** read AGENTS.md first. It is written for you.

The model, the vocabulary and the worked examples are public: riskmandate.ai/abp explains how it is sold and used; abp.sgit.ai holds the 23 capability primitives, the four barriers and the published deployment shapes this vault was computed against (v0.3.0).

Claude Code on the web, with one repository attached

For one agent in one deployment: everything it can do, what you authorised, the gap between the two, and what actually stands in the way.

TEMPLATE

Claude Code on the web, with one repository attached

Claude Code on the web (a remote session container) · grant 2026-09-05.2 · mandate 2026-09-09 · as at 2026-09-15

15

capabilities in the grant

6

wanted by the mandate

9

excess — granted, not asked for

7

unbounded — nothing in the way

13 of 15 rows measured 0 shortfall 6 aligned as stored in the vault

excess, by barrier



● none ● expectation ● setting ○ boundary — the only control

Looking for a score? There is none, on purpose. This document describes what the agent can do and what you asked for; it does not judge whether that is acceptable, because the same policy is harmless in one deployment and dangerous in another. The number you can move is the unbounded excess, and only a real control moves it.

The mandate, in one paragraph

I attached a repository and I want it worked on: read it, change it, run things, commit, and push to that repository — that is why I attached it. The container is disposable, so what it does to the container's own files is its business. I do not want it signing as me, and I do not want it creating sessions or routines that keep going after this one ends.

Elicited — starting-point, authored 2026-09-09. The only authored file in this vault.

The deployment

A managed cloud container, ephemeral, one git repository attached, an egress proxy above it, and a set of harness tools scoped by the platform. MEASURED, by the thing being profiled: the shell probed on 5 September with probes/run.py, the fetch tool's reach and the harness tools reported by the operator. HOST MEANS THE CONTAINER, not your machine; TENANT means the platform's scoped tokens, not your accounts. The same environment measured on 26 August is the Grant & Mandate library's first entry, and the two agree on every row they share.

Anthropic · surface agentbox · widest reach tenant

Four objects, and the verb attached to each

- **The mandate is elicited.** What the agent is authorised and expected to do — you already know it.
- **The grant is measured.** Everything the agent can do, from the shape, the account and the credentials.
- **The delta is derived.** Recomputed whenever either input moves; never edited by hand.
- **The barrier is recorded.** Per row, one of four kinds — and only the fourth is a control.

Validity

This describes the deployment shape as at this date. If the risk changed, the deployment changed — not this document.

- the grant version changes — a product release, a setting, a connector enabled or removed
- the mandate changes — the deployer authorises more or less
- the vocabulary version changes — a primitive is added, split or renamed
- a barrier moves — a setting becomes a boundary, or a boundary is removed

Delivered as an encrypted vault. This page is a little HTML on top of the files beside it, rendered straight from them. Anyone holding the read key can check every number here against the bytes it came from; the write side stays with the owner.

Everything the agent can do

15 capabilities, 13 measured. Irreversible rows first. Every row says how it is known, what stands in the way, and whether the effect can be undone.

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	V
authenticate-as.credential.signing	Sign commits with the key it holds	● none	no	observed ✓	S
delete.file.host	Delete files anywhere the account can reach	● none	no	observed ✓	S
read.credential.host	Read credentials stored where it runs	● none	no	observed ✓	S
read.file.host	Read any file the account can reach	● none	no	observed ✓	S
read.record.history	Read a retained record: shell history, past sessions	● none	no	observed ✓	S
authenticate-as.credential.tenant	Act in accounts with the credentials it holds	○ boundary	no	inferred	S (h (

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	V
					k t
send.endpoint.allowed	Reach a permitted list of hosts	○ boundary	no	observed ✓	s (f (h (k t
execute.process.host	Run programs as the account	● none	with-effort	observed ✓	s
write.file.host	Change any file the account can reach	● none	with-effort	observed ✓	s
write.file.project	Change the project it is working on	● none	with-effort	observed ✓	s
write.repository.project	Commit to the repository it was pointed at	● none	with-effort	observed ✓	s
write.repository.tenant	Push to a code host (any branch it can reach)	○ setting	with-effort	observed ✓	s (h (k t

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	V
read.file.project	Read the project it is working on	● none	yes	observed ✓	S (h (k t
create.schedule.tenant	Create something that outlives the session, on the platform (a routine, a scheduled trigger, a new session)	● setting	yes	self-reported	h (k t
create.schedule.host	Create something that outlives the turn where it runs (a cron, a service)	○ boundary	yes	observed ✓	S

What the words mean in this shape

- **host** — this container — ephemeral, the vendor's; not your machine
- **tenant** — the attached repository and the platform's scoped tokens; not your accounts
- **world** — the hosts the proxy allows

Not reachable from this shape

- **your machine's files** — the container has no path to the operator's computer; the assess tree records home as a boundary
- **your credentials** — no user credential is in the image; the keys present are the session's own
- **hosts the proxy refuses** — a 403 on the CONNECT, set above the process
- **repositories outside the platform's scope** — the token is scoped by the platform; the API tool refuses out-of-scope calls

The four barriers, and the test

	BARRIER	WHAT STANDS IN THE WAY	IS IT A CONTROL
●	none	nothing in the way	no
⦿	expectation	a rule in prose, enforced by nobody	no
⦿	setting	a switch the agent's own account can flip	no
○	boundary	enforced above the grant, out of the agent's reach	yes

A control bounds a grant only if it is enforced by something the grant does not include. A setting the agent's own account can change is not a control, because the grant includes the ability to remove the bound.

The notes behind the rows

- **authenticate-as.credential.signing** — commits are signed with the session's own key, registered as an agent identity in this site's registry (sha256-f9facb4c94da6c19) — not with yours
- **delete.file.host** — anything in the container, including the clone; irreversible for the container, and the container is disposable
- **read.credential.host** — the credential-shaped paths present are the SESSION'S OWN: its commit-signing key and its vault keystore. No user credential is in the container; presence cannot tell whose a key is, so this is the operator's account
- **read.file.host** — any file in the container — the attached clone, the harness's state, the system. Not your machine's files (the assess tree's 'home: boundary')
- **read.record.history** — the harness's project directory holds this session's own earlier tool outputs; no user shell history exists here

- **authenticate-as.credential.tenant** – five key-shaped variables and a code-host token — the platform's, scoped to in-scope repositories; it acts as the platform's app, never as you
- **send.endpoint.allowed** – six of six probed hosts answered through the proxy; a sibling container measured on 4 September had three refused: same product, two policies
- **execute.process.host** – root inside the container: every process and file IN THE CONTAINER. The container is the host; your machine is not reachable
- **write.file.host** – a zero-byte file was created and removed in /etc: system configuration of the container is writable
- **write.file.project** – the attached working tree is writable
- **write.repository.project** – a repository is attached and writable
- **write.repository.tenant** – the attached repository only (any branch it can reach); branch discipline is the clone's hooks, a setting; no rule at the host
- **read.file.project** – the attached working tree is readable
- **create.schedule.tenant** – a routine or a scheduled trigger resumes this session or spawns another later: it outlives the container
- **create.schedule.host** – systemctl and /etc/cron.d exist, so a cron can be written — and dies with the container; the real scheduler is the platform's routines, on the harness row

What you asked it to do — and what you did not

This is the draft asserting your mandate, and correcting it is how you state it. Three questions settle most rows: did you ask for it, would you object if it happened, or neither. Nothing here is sent anywhere; the app holds a read key and writes nothing.

In one paragraph

I attached a repository and I want it worked on: read it, change it, run things, commit, and push to that repository — that is why I attached it. The container is disposable, so what it does to the container's own files is its business. I do not want it signing as me, and I do not want it creating sessions or routines that keep going after this one ends.

6 wanted 9 excess 7 unbounded 0 shortfall as stored

Reset to the stored mandate

See the delta →

In the grant (15) — these are the rows that matter

CAPABILITY	WHAT IT IS	BARRIER	YOUR ANSWER
authenticate-as.credential.signing	Sign commits with the key it holds	● none	want do not want unsta
delete.file.host	Delete files anywhere the account can reach	● none	want do not want unstat

CAPABILITY	WHAT IT IS	BARRIER	YOUR ANSWER		
read.credential.host	Read credentials stored where it runs	● none	want do not want	do not want	unstat
read.file.host	Read any file the account can reach	● none	want do not want	do not want	unstat
read.record.history	Read a retained record: shell history, past sessions	● none	want do not want	do not want	unsta
authenticate-as.credential.tenant	Act in accounts with the credentials it holds	○ boundary	want do not want	do not want	unstat
send.endpoint.allowed	Reach a permitted list of hosts	○ boundary	want do not want	do not want	unsta
execute.process.host	Run programs as the account	● none	want do not want	do not want	unsta
write.file.host	Change any file the account can reach	● none	want do not want	do not want	unstat

CAPABILITY	WHAT IT IS	BARRIER	YOUR ANSWER		
write.file.project	Change the project it is working on	● none	want do not want unsta		
write.repository.project	Commit to the repository it was pointed at	● none	want do not want unsta		
write.repository.tenant	Push to a code host (any branch it can reach)	● setting	want do not want unsta		
read.file.project	Read the project it is working on	● none	want do not want unsta		
create.schedule.tenant	Create something that outlives the session, on the platform (a routine, a scheduled trigger, a new session)	● setting	want do not want unsta		
create.schedule.host	Create something that outlives the turn	○ boundary	want do not want unstat		

CAPABILITY	WHAT IT IS	BARRIER	YOUR ANSWER
	where it runs (a cron, a service)		

Not in this grant (8) — stated for completeness

CAPABILITY	WHAT IT IS	REACH	YOUR ANSWER		
execute.process.self	Run programs inside its own sandbox only	self	want	do not want	unstated
send.endpoint.world	Reach any host on the internet	world	want	do not want	unstated
grant.credential.self	Change its own permission settings	self	want	do not want	unstated
send.message.world	Send a message to anyone	world	want	do not want	unstated
read.message.tenant	Read mail or chat it is connected to	tenant	want	do not want	unstated
create.record.world	Publish packages, images or pages under the name it holds	world	want	do not want	unstated
write.budget.tenant	Spend money or tokens against an account it holds	tenant	want	do not want	unstated
read.record.browsing	Read every page you visit	host	want	do not want	unstated

Your corrected mandate, as a file

Paste this over data/mandate.json in the vault and rebuild; the delta is recomputed and every document is rewritten. A hand-edited delta is undone by the next build, which is the point.

Copy

2,495 characters

```
{  
  "type": "abp/mandate/v1",  
  "id": "coding-assistant-in-a-container",  
}
```

What it can do that you did not ask for

As stored in the vault: computed 2026-09-15T00:17:26Z from grant 2026-09-05.2 and mandate 2026-09-09.

AS STORED

9

excess

7

unbounded excess

0

shortfall — wanted, not granted

6

aligned

Excess you refused (3)

You said no. The last column says what, if anything, enforces the no.

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHA
authenticate-as.credential.signing	Sign commits with the key it holds	● none	no	observed ✓	—
read.record.history	Read a retained record: shell history, past sessions	● none	no	observed ✓	—
create.schedule.tenant	Create something that outlives the session, on the platform (a routine, a scheduled trigger, a	● setting	yes	self-reported	the are list

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHAT IT IS
	new session)				

Excess you never mentioned (6)

Authority nobody scoped. Not wrong — unstated. These are the rows to read twice.

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHAT IT IS
delete.file.host	Delete files anywhere the account can reach	● none	no	observed ✓	—
read.credential.host	Read credentials stored where it runs	● none	no	observed ✓	—
read.file.host	Read any file the account can reach	● none	no	observed ✓	—
authenticate-as.credential.tenant	Act in accounts with the credentials it holds	○ boundary	no	inferred	th by sc
write.file.host	Change any file the account can reach	● none	with-effort	observed ✓	—
create.schedule.host	Create something that outlives the turn	○ boundary	yes	observed ✓	th ep sc wi

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WH
	where it runs (a cron, a service)				

Unbounded excess (7)

The excess whose barrier is anything but a boundary. Every real control moves one row out of this list; nothing else does.

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WH
authenticate-as.credential.signing	Sign commits with the key it holds	● none	no	observed ✓	—
delete.file.host	Delete files anywhere the account can reach	● none	no	observed ✓	—
read.credential.host	Read credentials stored where it runs	● none	no	observed ✓	—
read.file.host	Read any file the account can reach	● none	no	observed ✓	—
read.record.history	Read a retained record: shell history, past sessions	● none	no	observed ✓	—

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHAT STANDS
write.file.host	Change any file the account can reach	● none	with-effort	observed ✓	—
create.schedule.tenant	Create something that outlives the session, on the platform (a routine, a scheduled trigger, a new session)	○ setting	yes	self-reported	the ar lis

Shortfall (0)

none — everything wanted is granted.

Aligned (6)

Wanted and granted.

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHAT STANDS
send.endpoint.allowed	Reach a permitted list of hosts	○ boundary	no	observed ✓	a mandatory proxy config this process refuses are with a 403 c CONNECT; t probed on 5 all answered
execute.process.host	Run programs as the account	● none	with-effort	observed ✓	—

CAPABILITY	WHAT IT IS	BARRIER	UNDO	EVIDENCE	WHAT STANDS
write.file.project	Change the project it is working on	● none	with-effort	observed ✓	—
write.repository.project	Commit to the repository it was pointed at	● none	with-effort	observed ✓	—
write.repository.tenant	Push to a code host (any branch it can reach)	● setting	with-effort	observed ✓	pre-commit push hooks (the mandat the insuranc refuse by ex no-verify pa branch rule
read.file.project	Read the project it is working on	● none	yes	observed ✓	—

The organisation authorises the agent, under this behaviour policy, for an interval

The organisation is the authority, the behaviour policy is the instrument, the agent is the licensee. Scope, conditions, interval, signature — and each condition next to what enforces it.

Licensee	Claude Code on the web (a remote session container) — shape anthropic/claude-code-remote/ccr-container, grant 2026-09-05.2
Authority	— not yet issued to anyone —
Accountable owner	— unassigned —
Instrument	mandate coding-assistant-in-a-container (2026-09-09), delta anthropic__claude-code-remote__ccr-container__coding-assistant-in-a-container, vocabulary abp.sgit.ai v0.3.0
Issued	— not issued: this is a template —
Valid until	— an interval is set when it is issued; a licence with no expiry is not a decision —

Scope — authorised (6)

- `read.file.project` — Read the project it is working on
- `write.file.project` — Change the project it is working on
- `execute.process.host` — Run programs as the account
- `write.repository.project` — Commit to the repository it was pointed at
- `write.repository.tenant` — Push to a code host (any branch it can reach)
- `send.endpoint.allowed` — Reach a permitted list of hosts

Conditions — asked not to, and what enforces each (9)

Conditions with ● or ● beside them are asked, not enforced; the organisation issuing this licence is accepting that, for the interval above, with its eyes open.

CONDITION	ASKED BECAUSE	BARRIER	ENFORCED BY
Do not authenticate-as.credential.signing — sign commits with the key it holds	the mandate refuses it	● none	nothing — a line in prose

CONDITION	ASKED BECAUSE	BARRIER	ENFORCED BY
Do not delete.file.host — delete files anywhere the account can reach	the mandate never authorised it	● none	nothing — a line in prose
Do not read.credential.host — read credentials stored where it runs	the mandate never authorised it	● none	nothing — a line in prose
Do not read.file.host — read any file the account can reach	the mandate never authorised it	● none	nothing — a line in prose
Do not read.record.history — read a retained record: shell history, past sessions	the mandate refuses it	● none	nothing — a line in prose
Do not authenticate-as.credential.tenant — act in accounts with the credentials it holds	the mandate never authorised it	○ boundary	the token's scope, set by the platform (in-scope repositories only)
Do not write.file.host — change any file the account can reach	the mandate never authorised it	● none	nothing — a line in prose
Do not create.schedule.tenant — create something that outlives the session, on the platform (a routine, a scheduled trigger, a new session)	the mandate refuses it	● setting	the platform's routines are the operator's to list and delete
Do not create.schedule.host — create something that outlives the turn where it runs (a cron, a service)	the mandate never authorised it	○ boundary	the container is ephemeral: whatever is scheduled here dies with it

Void when

- the grant version changes — a product release, a setting, a connector enabled or removed
- the mandate changes — the deployer authorises more or less
- the vocabulary version changes — a primitive is added, split or renamed

- a barrier moves — a setting becomes a boundary, or a boundary is removed

When any of those happens the deployment changed, not this document.
Rebuild, re-read the delta, re-issue.

Signature

Unsigned. A template is not issued to anybody. When this vault is customised for one organisation, a named person signs here, with the date and the interval, and the same person owns the review when the interval ends.

Three ways in. The same files each time.

The policy is a set of markdown files the agent reads. Paste them, download them, or print them — whichever fits how the agent is run. AGENTS.md goes wherever the agent already reads its instructions; the rest sit beside it.

01 · COPY AND PASTE

Blocks, per file or all at once

For a session you are starting now, or a CLAUDE.md you already have. Below.

02 · THE ZIP

Every file, ready to unpack

The markdown, the JSON, the vocabulary and the history — the whole vault minus this app.

[Download the zip](#)

Built by the same script that renders the documents, so it cannot disagree with them.

03 · THE PDF

The whole policy, printed

For the person who wants to read it on paper, or hand it to a board — every view of this app, in order.

[Download the PDF](#)

Rendered from this app in print mode, dated in the footer.

Everything, in one prompt

The seven files concatenated, each under its own marker. Paste it into a fresh session, or into the file the agent reads first.

[Copy](#) 41,151 characters

```
<!-- AGENTS.md -->
# AGENTS.md — how to treat the behaviour policy files beside this one
```

One file at a time, with where it goes

AGENTS.md

Append to CLAUDE.md, AGENTS.md or a ROLE.md — or drop it into a fresh session first. It tells the agent what the other files are and what to do with them.

SKILL.md

For a skills directory. The same instructions in the portable skill format, with the format's limit stated on its face.

MANDATE.md The scope. Beside AGENTS.md, always.

GRANT.md What it can reach, with a barrier per row. A description, not permission.

DELTA.md What nobody asked for — the list the agent stops and reports before touching.

LICENCE-TO-OPERATE.md Who authorised it, for what, until when, on what conditions.

AGENT-BEHAVIOUR-POLICY.md

All four objects in one document, if one file is all there is room for.

What these files cannot do, in their own words. AGENTS.md is a rule in prose — the second of four barriers — and it bounds nothing. It shapes what the agent tends to do; it does not change what it can do. What would is in the Barrier column of the grant: a token scope, a branch rule, a sandbox, an egress proxy. Most of what goes wrong is not an attack, and for that these files are the telling.

What this describes, and what it was computed from

One history entry per recompute whose counts moved. The delta pins the version of both inputs and the time it was computed, so a later build can say what moved.

Pinned inputs

- **Grant** anthropic/claude-code-remote/ccr-container · version 2026-09-05.2 · 13 of 20 rows measured
- **Mandate** coding-assistant-in-a-container · starting-point · authored 2026-09-09
- **Delta** anthropic__claude-code-remote__ccr-container__coding-assistant-in-a-container · computed 2026-09-15T00:17:26Z · riskmandate.ai build-abp-vault.mjs (abp.delta/v1 semantics)
- **Vocabulary** abp.sgit.ai v0.3.0 — 23 capabilities, 4 barriers, 3 undo classes
- **As at** 2026-09-15

Where this page read its data

From files beside this page — 19 fetched. Outside a vault host this is a static copy; the vault is the one that is right.

The app holds a read key at most and declares no write permission, so nothing here can change the vault.

History

v1 2026-09-15T00:17:26Z · grant 2026-09-05.2 · mandate 2026-09-09 · excess 9, unbounded 7, shortfall 0, aligned 6

Provenance

Promoted from profiles/anthropic/claude-code-remote/ccr-container.json. The grant is the union of its tool rows; where two tools reach the same capability the WEAKEST barrier is kept, because the agent takes the easier path. Source: <https://what-can-it-do.games.sgit.ai/map/index.html>, retrieved 2026-09-11T13:00:37Z, pack v0.8.0, sha256:d6d4ba40f1fb1f93f660687e4787ac10c2e1835efeb3929a4c8ad62cee8897ef. This describes a published deployment shape. It is not an assessment, an audit, a

certification or a security review of any named product, and it carries no adjective and no score.

README

```
# Claude Code on the web, with one repository attached – an Agent Behaviour Policy, as a vault
```

```
> This is the template vault for one deployment shape: the draft a buyer corrects. Everything in it is derived from three inputs, and one of them is yours.
```

```
**Vault** `claude-code-web` · **status** template · **shape** `anthropic/claude-code-remote/ccr-container` · **grant** 2026-09-05.2 · **mandate** 2026-09-09 · **vocabulary** abp.sgit.ai v0.3.0 · **as at** 2026-09-15
```

```
---
```

```
## What is in here
```

```
| File | What it is | Who writes it |
| --- | --- | --- |
| `AGENT-BEHAVIOUR-POLICY.md` | The four objects in one document: grant, mandate, delta, barrier | derived |
| `MANDATE.md` · `data/mandate.json` | What the agent is authorised and expected to do | **you** – the only authored file |
| `GRANT.md` · `data/grant.json` | Everything the agent can do, one row per capability, with what stands in the way of each | measured from the shape; never typed |
| `DELTA.md` · `data/delta.json` | Excess, shortfall, and the part of the excess nothing bounds | derived, never authored, recomputed on every build |
| `LICENCE-TO-OPERATE.md` | The organisation authorises the agent to operate under this behaviour policy, for an interval, with conditions | derived from the mandate; signed by a named person when issued |
| `AGENTS.md` | Drop this into the agent's own context – `CLAUDE.md`, `AGENTS.md`, a `ROLE.md`, a skill – so it knows how to treat the rest | generic; travels unchanged |
| `SKILL.md` | The same, in the portable agent-skill format | generic; travels unchanged |
| `data/validity.json` | What this describes, as at when, and what would void it | derived |
| `data/vocabulary/` | The 23 capability primitives, 4 barriers, 3 undo classes and evidence tiers this was computed against, pinned | copied from abp.sgit.ai, versioned |
| `history/` | One entry per recompute whose counts moved | derived |
```

```
## How to read it
```

1. `**`MANDATE.md` first**`, because it is the one thing you already know. If it is wrong, it is wrong upward – most people authorised less than the draft assumes.
2. `**`GRANT.md` second.**` Every row says how it is known (measured or derived), what stands in the way (one of four barriers), and whether the effect can be undone.
3. `**`DELTA.md` third.**` The list of what it can do and you did not ask for, split into the part you refused, the part you never mentioned, and the part nothing bounds.
4. `**`LICENCE-TO-OPERATE.md` last.**` What the organisation is actually authorising, for how long, on what conditions, and who signs.

How to correct it

Edit ``data/mandate.json`` – move a capability between ``want``, ``do_not_want`` and ``unstated`` – and rebuild:

```
...  
node scripts/site/build-abp-vault.mjs claude-code-web  
...
```

The delta is recomputed and the documents are rewritten. Nothing else in this vault is edited by hand, and a hand-edited delta would be detected: the build recomputes it from the inputs every time.

How to give it to the agent

Copy ``AGENTS.md`` into the agent's working context (or append it to the file the agent already reads – ``CLAUDE.md``, ``AGENTS.md``, ``ROLE.md``, a ``SKILL.md``), and put ``MANDATE.md``, ``GRANT.md`` and ``AGENT-BEHAVIOUR-POLICY.md`` beside it. ``AGENTS.md`` tells the agent what each file is, what to do with it, and – in its own words – what a file like that cannot do.

What this is not

- Not a risk assessment. It has no assets in it and no consequences; it is the input to one.
- Not a compliance assessment, a certification, an audit or a security review of any named product.
- Not a guardrail. The barrier column in ``GRANT.md`` says which rows are bounded by something and which are only asked.
- Not scored. A behaviour policy cannot be dangerous; a deployment can.

`_`This describes the deployment shape as at this date. If the risk changed, the deployment changed – not this document.`_`

No score, rating, level or traffic light appears in this vault or in its data, and none will. The behaviour policy describes; it does not judge.

Generated by ``scripts/site/build-abp-vault.mjs`` from ``data/grant.json``, ``data/mandate.json`` and the pinned vocabulary; ``data/mandate.json`` is the only

file a person writes. Licence: CC BY 4.0.

RiskMandate — Agent Behaviour Policy, delivered as a vault. Grant 2026-09-05.2 · mandate 2026-09-09 · vocabulary abp.sgit.ai v0.3.0 · as at 2026-09-15. It describes and it does not judge, so it carries no score. Not a compliance assessment, a certification, an audit or a security review of any named product. CC BY 4.0. [riskmandate.ai](#) · [abp.sgit.ai](#)